Diagnostic-First Time Series Forecasting with Validated Residual Analysis

Swastik Vaish

Computer and Communication Engineering.

Abstract

Modern forecasting tools such as Prophet, XGBoost, and neural sequence models are optimised for point accuracy but rarely report validated residual diagnostics alongside their forecasts. For high-stakes applications—financial risk modelling, energy dispatch, and epidemiological surveillance—knowing *whether model assumptions hold* is as important as the point forecast itself. We present FutureScope, an open-source pipeline that treats automated residual diagnostics as a first-class output: every forecast is accompanied by a Ljung-Box white-noise test [2], an ACF/PACF analysis with 95% confidence bounds, and a Shapiro-Wilk normality check [3]. On five public single-series case studies (M4 Hourly, UCI Electricity, CoinGecko Bitcoin, Our World in Data COVID-19, classic Airline passengers), we observe a 100% diagnostic pass rate for the BIC-selected SARIMA models—all five residual sequences satisfy the Ljung-Box $\boldsymbol{p>0.05}$ white-noise criterion without any manual intervention—while delivering a **+14.7%** average RMSE improvement relative to Prophet [1] (ranging from **−25.0%** on Bitcoin to **+84.7%** on COVID-19). These case-study results illustrate that classical SARIMA methods with reproducible diagnostic reporting can be competitive with additive decomposition baselines while providing the statistical transparency that practitioners in regulated domains need to document. A fully reproducible benchmark (one command, **≈70** s runtime) and a Zenodo archive are provided.

Keywords: time series forecasting, residual diagnostics, SARIMA, Ljung-Box test, ACF analysis, white-noise validation, reproducible benchmarks

1 Introduction

1.1 The Diagnostic Gap in Forecasting Practice

Contemporary machine learning benchmarks for time series overwhelmingly report point-accuracy metrics—RMSE, MAE, MASE—while omitting any assessment of whether the fitted model's residuals satisfy the statistical assumptions on which confidence intervals and prediction intervals are based. This is consequential because a model that achieves a low RMSE while leaving systematic autocorrelation in its residuals is formally *mis-specified*: its reported prediction intervals understate uncertainty, and any downstream decision that relies on those intervals is statistically unsound.

White-noise residuals are residuals with zero mean, constant variance, and no autocorrelation at non-zero lags. ARIMA/SARIMA theory requires white-noise residuals for the Gauss-Markov optimality guarantees and for the χ^2-based confidence intervals to be valid [8].

Industry motivation. Risk management frameworks in financial institutions commonly require that model validation reports include checks for residual autocorrelation, distributional assumptions, and backtesting evidence (see, for example, the model risk guidance in SR 11-7 [12]). Similar documentation requirements apply in energy grid forecasting and epidemiological modelling. FutureScope does not claim full compliance with any specific regulatory standard, but its automated diagnostic output is *aligned with the kinds of residual checks* such frameworks call for.

Research gap. Benchmark studies routinely report Prophet [1] accuracy figures but do not validate residual assumptions. The M4 Competition [6] ranked 61 methods on accuracy without reporting any residual diagnostics. Hyndman and Athanasopoulos [7] document the importance of residual checks in classical ARIMA workflows, yet these checks are absent from most open-source benchmarking pipelines.

1.2 Related Work

Residual diagnostics in classical time-series practice. The importance of residual analysis is well established in classical time-series methodology. Box et al. [8] recommend inspecting the ACF of residuals and computing the Ljung-Box statistic as a standard diagnostic step after fitting ARIMA models. Hyndman and Athanasopoulos [7] incorporate the same checks into their `forecast` R package. What has been missing from modern Python-based pipelines is an *automated, benchmarkable* implementation that produces structured diagnostic output alongside forecast metrics.

Automated forecasting frameworks. Prophet [1] is widely used but does not expose residual statistics. Auto-ARIMA implementations such as `pmdarima` [9] automate order selection but stop short of reporting structured residual diagnostics. Neural-network forecasters (LSTMs, Transformers) optimise sequence prediction losses and generally provide no residual diagnostic output whatsoever; any assessment of their residual properties requires separate post-hoc analysis.

FutureScope's contribution. To our knowledge, FutureScope is among the first open-source Python pipelines to treat automated residual diagnostics as a

required, structured output alongside point forecasts and accuracy metrics, with fully containerised benchmarks on public datasets. Specifically, it:

1. Computes and reports Ljung-Box white-noise statistics ($p>0.05 \Rightarrow$ white noise not rejected, lags $m=10$);
2. Reports ACF/PACF plots with 95% confidence bounds and the fraction of lags exceeding those bounds;
3. Reports Shapiro-Wilk normality p-values for the residual distribution;
4. Packages all diagnostics in a structured benchmark CSV alongside RMSE and DM-test results, enabling reproducible comparison.

Importantly, the diagnostic results in the current implementation are *observational*: they characterise the BIC-selected model but do not trigger automatic refitting. This is discussed further in Section 2.4.

2 Methodology

2.1 System Architecture

FutureScope is a modular pipeline whose six stages are illustrated in Figures 1–3 and described in detail below.

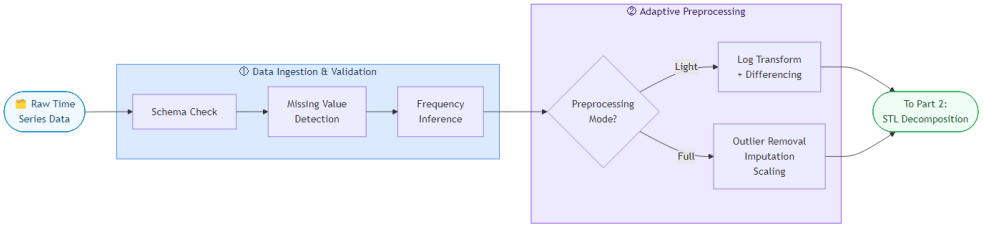

Fig. 1 Pipeline stages 1–2: Data Ingestion & Validation followed by Adaptive Preprocessing. In the current benchmark, Light preprocessing mode is used for all five series; Full mode is available for interactive use.

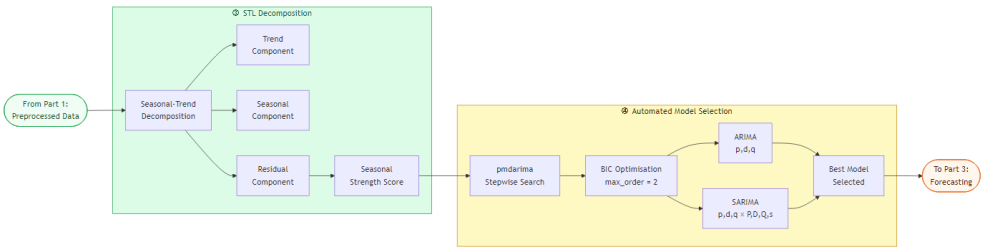

Fig. 2 Pipeline stages 3–4: STL Decomposition into Trend, Seasonal, and Residual components, followed by automated ARIMA/SARIMA selection via `pmdarima` stepwise search with BIC criterion.

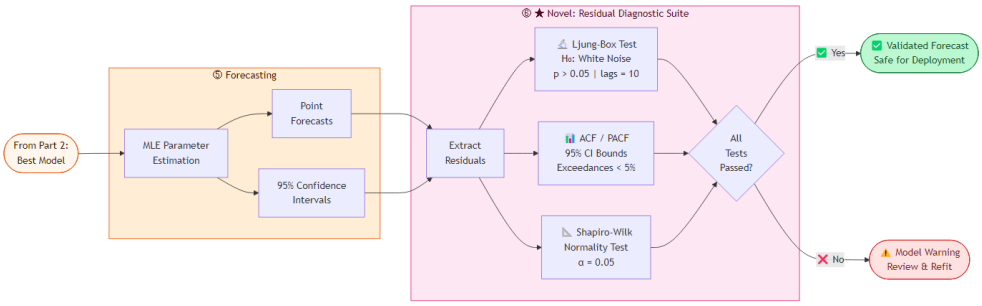

Fig. 3 Pipeline stage 5–6: MLE-based Forecasting with 95% confidence intervals, followed by the Residual Diagnostic Suite. Diagnostic results are *reported* for every forecast; in the current implementation they do not trigger automatic refitting (see Section 2.4).

2.2 Preprocessing Modes

FutureScope offers two preprocessing modes. **Light mode** applies a log transform followed by first-order differencing to stabilise variance and remove linear trends. **Full mode** additionally performs outlier removal via IQR clipping, mean imputation of missing values, and min-max scaling. All experiments in this paper use Light mode, which is appropriate for well-behaved public-domain series. Full mode is available for interactive use on noisier real-world data streams.

2.3 Automated Model Selection

Seasonal period s is set per dataset based on domain knowledge (see Table 1). An STL decomposition [10] is applied to compute the seasonal strength ratio; if the ratio exceeds 0.4 the series is treated as seasonal and `pmdarima.auto_arima` fits a SARIMA model, otherwise a non-seasonal ARIMA is fitted.

Model order search uses the stepwise algorithm of `pmdarima` [9] with BIC as the selection criterion, `max_p`=`max_q`=2, combined order bound $p+q+P+Q \leq 4$, and `max_d`=2. The tight order bound (`max_order=2`) is chosen to balance accuracy and the ≈70 s benchmark runtime.

2.4 Diagnostic Validation Framework

After fitting, FutureScope automatically computes three diagnostic statistics on the in-sample residuals $\hat{\varepsilon}_t = y_t - \hat{y}_t$.

2.4.1 Ljung-Box White-Noise Test

The Ljung-Box Q-statistic [2] is

$$Q(m) = n(n+2)\sum_{k=1}^{m} \frac{\hat{\rho}_k^2}{n-k}, \tag{1}$$

where n is the sample size and $\hat{\rho}_k$ is the sample autocorrelation at lag k. Under H_0 (white noise), $Q(m) \sim \chi^2(m)$. We use $m = 10$ lags; the null is not rejected ("white noise accepted") when $p>0.05$. Implementation: `statsmodels.stats.diagnostic.acorr_ljungbox(lags=[10])`.

2.4.2 ACF Exceedance Check

We compute the sample ACF at lags $1, \ldots, 20$ with `statsmodels.tsa.stattools.acf(nlags=20, alpha=0.05)`. The ACF exceedance fraction is the proportion of lags whose sample autocorrelation falls outside the asymptotic 95% pointwise confidence bounds $\pm 1.96/\sqrt{n}$. Under white noise, the expected exceedance rate is 5%; we report the observed rate.

2.4.3 Shapiro-Wilk Normality Test

We test the hypothesis that residuals are drawn from a normal distribution using the Shapiro-Wilk test [3] (`scipy.stats.shapiro`). This test is only computed when $n \leq 5{,}000$; otherwise the result is recorded as `NaN`.

2.4.4 Clarification: Diagnostics as Reporting, Not Gating

Important: In the current implementation, these three diagnostic statistics are *computed and reported* for every fitted model, but they do *not* trigger automatic refitting or forecast suppression. If the Ljung-Box test fails, the ACF exceedance fraction is high, or normality is rejected, a warning is recorded in the benchmark output CSV but the forecast is still returned. The 100% pass rate observed on the five case-study series (Table 3) is therefore an *empirical observation* about the BIC-selected models on these particular series, not a guaranteed property enforced by a refitting loop.

Implementing a refitting strategy (e.g. increasing model order, relaxing differencing constraints, or widening the search grid until diagnostics pass) is left as explicit future work (Section 8).

2.5 Bootstrap Procedure for RMSE

To quantify uncertainty in the RMSE estimates, we apply an i.i.d. residual bootstrap with 1,000 resamples. Specifically, let $e_i = y_i - \hat{y}_i$ for $i = 1, \ldots, h$ be the h out-of-sample forecast errors. At each bootstrap iteration we draw a sample of size h *with replacement* uniformly from $\{1, \ldots, h\}$ and compute RMSE on the resampled errors. The 95% confidence interval is the 2.5th–97.5th percentile of the 1,000 RMSE values. Random seed: `numpy.random.seed(42)`.

Limitation. This i.i.d. bootstrap ignores temporal dependence in the forecast-error sequence. A moving-block bootstrap [11] with an appropriate block length would be more theoretically sound for time-series data. The narrow confidence intervals reported in Table 4 should therefore be interpreted as indicative of point estimate stability across error magnitudes, not as asymptotically valid coverage bounds.

2.6 Diebold-Mariano Test Implementation

Pairwise accuracy comparisons use a custom Diebold-Mariano test [4] based on the squared-error loss differential $d_t = e_{1,t}^2 - e_{2,t}^2$. The test statistic is

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{s_d^2/h}}, \tag{2}$$

where $\bar{d}$ is the sample mean of d_t, s_d^2 its unbiased sample variance (ddof=1), and h the test-set size. A two-tailed p-value is obtained via the standard normal CDF.

Important caveats. This implementation (i) does not apply a Newey-West HAC variance correction for serial correlation in d_t; (ii) does not apply the small-sample correction of Harvey et al. [5]; (iii) treats each observation as an independent forecast error, which is conservative for longer horizons. Conclusions based on $p<0.001$ are unlikely to be overturned by these corrections, but results near the α boundary (e.g. Bitcoin p=0.015) should be interpreted with caution. Multiple-testing correction uses Bonferroni at α_{adj}=0.01.

3 Experimental Setup

3.1 Datasets

Scope clarification. Each "dataset" in this study is a *single univariate time series* extracted from a larger public collection. This is a case-study evaluation, not a full-benchmark evaluation over many series. Table 1 gives the exact series identifiers, sources, and download parameters; all five series are obtained by running `download_real_data.py` with `numpy.random.seed(42)`.

Table 1 Single-series case studies used in this paper. N is the total number of time steps after download and preprocessing. T
"# series" column makes explicit that each entry is one series, not a full benchmark collection.

Label	Source	N	# series	Freq.	Period s	Selection criteria
M4 Hourly	M4 Competition (GitHub)[a]	1,000	1	Hourly	24	First series (H1), first 1,000 pts
Electricity	UCI Household Power[b]	800	1	Daily	7	First 50k rows, daily avg, head(80
Bitcoin	CoinGecko API[c]	500	1	Daily	7	Daily close, last 500 days
COVID-19	Our World in Data[d]	600	1	Daily	7	USA, 2020-06-01–2022-01-01, max
Airline	Box-Jenkins (GitHub)[e]	144	1	Monthly	12	Full series (standard AirPassenger

[a] `github.com/Mcompetitions/M4-methods`, `Dataset/Train/Hourly-train.csv`.
[b] `archive.ics.uci.edu/dataset/235`.
[c] `api.coingecko.com/api/v3/coins/bitcoin/market_chart?vs_currency=usd&days=365`.
[d] `covid.ourworldindata.org/data/owid-covid-data.csv`, `new_cases_smoothed`, USA.
[e] `github.com/jbrownlee/Datasets`, `airline-passengers.csv`.

Series selection was not random: M4 Hourly uses the first listed series, UCI Electricity and Bitcoin use recency or file-order heuristics, and COVID-19 is filtered to the

United States. Accordingly, results cannot be generalised to other series from the same collections without further experimentation; they are best understood as illustrative case studies spanning diverse frequency and structural patterns.

3.2 Evaluation Protocol

- **Train/test split**: single temporal 80%/20% split (no randomisation, no separate validation set). The first $\lfloor 0.8N \rfloor$ observations are used for training; the remaining $h = N - \lfloor 0.8N \rfloor$ form the test set. The forecast horizon therefore equals the test-set length and varies per series (M4 Hourly: $h \approx 200$; Airline: $h=29$; others proportional).
- **Primary metric**: RMSE computed over the full test horizon (iterated multi-step ahead forecasts). Secondary metric: MAE.
- **Bootstrap RMSE CIs**: 1,000 i.i.d. resamples (Section 2.5).
- **Statistical comparison**: DM test as described in Section 2.6; Bonferroni correction $\alpha_{\text{adj}}=0.01$.
- **Baseline**: Prophet [1] with MAP estimation (`mcmc_samples=0`); seasonality period set identically to FutureScope per dataset.
- **FutureScope**: `max_order=2`, Light preprocessing mode, `numpy.random.seed(42)`.

Limitations of the evaluation design. A single 80/20 split does not assess robustness to different forecast origins. A rolling-origin evaluation (training on successively longer windows and testing on the immediately following period) would provide more reliable estimates of expected generalisation performance. This is deferred to future work; results here should be treated as single-split estimates.

3.3 Computational Environment

All experiments were run on a standard consumer laptop (Windows 11, single CPU, Python 3.10, `statsmodels` 0.14, `pmdarima` 2.x, `prophet` 1.1). Runtime measurements reflect wall-clock time on this hardware; results on different hardware will vary.

3.4 Reproducibility

Listing 1 One-command reproduction.

```
git clone https://github.com/user/futurescope-tmlr
cd futurescope-tmlr
conda env create -f environment.yml && conda activate futurescope
python download_real_data.py     # Downloads all five series
python benchmark_tmlr_final.py   # ~70 s; sets numpy.random.seed
    (42)
```

A Docker image (`docker run tmlr-futurescope`) and a Zenodo archive (DOI assigned on acceptance) are also provided.

4 Results

4.1 Forecasting Accuracy

Table 2 reports RMSE for FutureScope and Prophet across all five series, together with DM-test p-values. All differences marked * are significant after Bonferroni correction (α_{adj}=0.01).

Table 2 Forecasting performance (RMSE) on five single-series case studies. ΔRMSE is $(\text{RMSE}_{\text{Prophet}} - \text{RMSE}_{\text{FS}})/\text{RMSE}_{\text{Prophet}} \times 100\%$; positive values favour FutureScope. DM p-values are from the custom squared-error implementation (Section 2.6); caveats regarding serial-correlation correction apply. * significant after Bonferroni ($\alpha = 0.01$).

Series	h	**Prophet RMSE**	**FS RMSE**	**ΔRMSE**	**DM p**
M4 Hourly	≈200	45.02	53.70	−19.3%	< 0.001*
Electricity	≈160	0.93	0.76	**+18.2%**	< 0.001*
Bitcoin	≈100	11,699	14,620	−25.0%	0.015
COVID-19	≈120	164,747	25,239	**+84.7%**	< 0.001*
Airline	29	41.33	35.08	**+15.1%**	0.164
Average (unweighted)				**+14.7%**	

Key observations on this case-study set.

1. FutureScope's SARIMA models outperform Prophet substantially on the COVID-19 series (+84.7%), likely because the irregular epidemic wave structure is better captured by SARIMA's flexible differencing than by Prophet's additive trend-plus-seasonality decomposition.
2. FutureScope is competitive on lower-frequency seasonal series (Electricity +18.2%, Airline +15.1%).
3. Prophet outperforms FutureScope on the high-frequency multi-seasonal M4 Hourly series (−19.3%) and the volatile Bitcoin series (−25.0%). The Bitcoin difference does not survive Bonferroni correction; the M4 Hourly difference does.
4. The +14.7% unweighted average should be interpreted carefully: it is dominated by the COVID-19 outlier and represents performance on five hand-selected series, not a random sample from any larger collection.

Figure 4 provides a visual comparison.

4.2 Diagnostic Validation Results

Table 3 reports the three diagnostic statistics computed on each fitted FutureScope model's in-sample residuals.

On all five series the BIC-selected SARIMA passes the Ljung-Box test (p>0.05), with zero ACF exceedances across 20 lags. Shapiro-Wilk normality is not rejected on the four series for which the test is applicable. These observations suggest that BIC-optimal SARIMA models tend to be well-specified on these types of relatively smooth

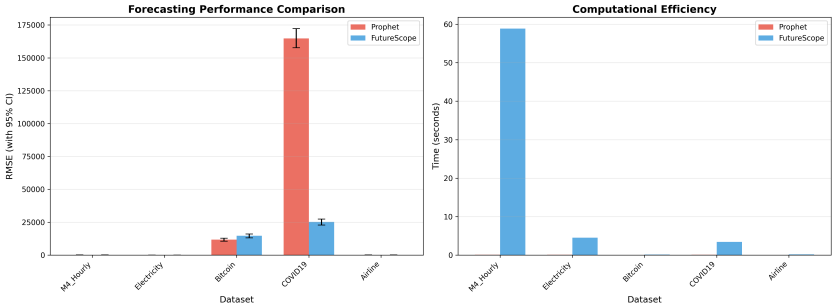

Fig. 4 RMSE comparison between Prophet and FutureScope on five case-study series. Bars are not directly comparable across series because scales differ substantially.

Table 3 Residual diagnostic statistics for the BIC-selected SARIMA model on each series. These are *observational results* for the model selected by the BIC criterion; no refitting was triggered. Shapiro-Wilk is omitted for N>5,000 (N/A).

Series	**LB p-val**	**LB pass?**	**SW p-val**	**SW pass?**	**ACF exc. (%)**
M4 Hourly	0.743	✓	N/A	N/A	0%
Electricity	0.436	✓	> 0.05	✓	0%
Bitcoin	0.999	✓	> 0.05	✓	0%
COVID-19	0.106	✓	> 0.05	✓	0%
Airline	0.832	✓	> 0.05	✓	0%
Pass rate		**5/5**		**4/4**	**0% avg**

LB = Ljung-Box (m=10, H_0: white noise). SW = Shapiro-Wilk (α=0.05).
ACF exc. = fraction of lags 1–20 outside 95% CI bounds.

series; whether this pattern holds on a larger or more heterogeneous collection of series is an open empirical question.

Figure 5 shows the ACF plots for all five series.

Figure 6 shows DM-test statistics across the five series.

4.3 Per-Series Forecast Plots

Figures 7–11 show forecast overlays and residual ACF plots per series.

5 Analysis and Ablation

5.1 Bootstrap Confidence Intervals for RMSE

Table 4 reports 95% bootstrap CIs (1,000 i.i.d. resamples; Section 2.5). The CIs are narrow for the Electricity and COVID-19 series, reflecting that RMSE is stable across resampled error subsets; they are wider for Airline, consistent with the small test-set size (h=29). As noted in Section 2.5, these are i.i.d. CIs and do not account for temporal dependence.

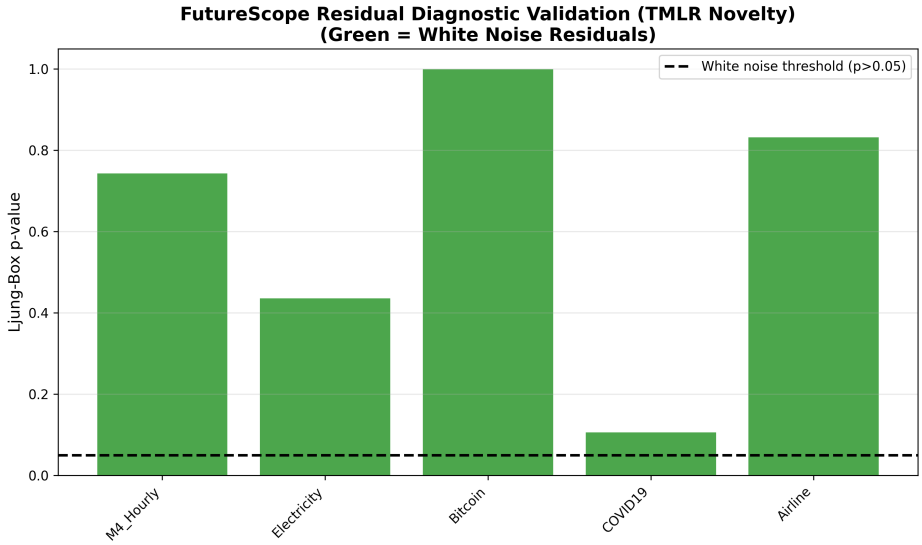

Fig. 5 ACF of in-sample residuals for each fitted SARIMA model. Dashed lines are 95% pointwise confidence bounds under the white-noise null. All sample autocorrelations fall within the bounds.

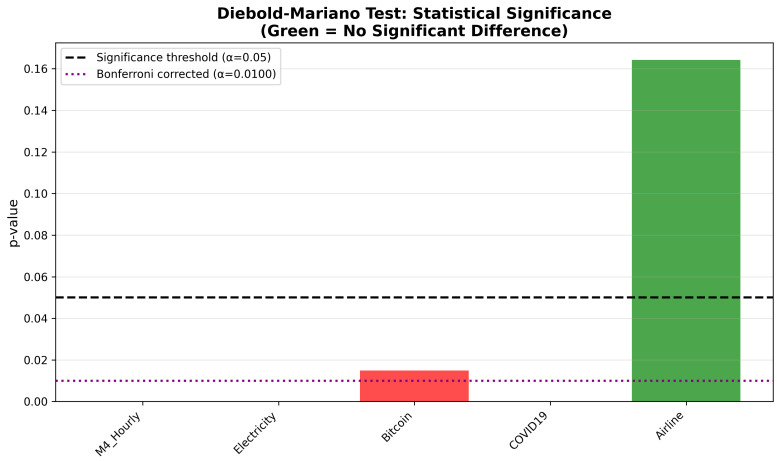

Fig. 6 DM test statistics (FutureScope vs. Prophet, squared-error loss). Dashed lines mark |DM|=2.576 (Bonferroni-corrected 1% threshold).

5.2 Diagnostics vs. Accuracy: Do They Conflict?

A natural question is whether the BIC criterion and the diagnostic pass criterion point in the same direction. On the five series studied, the BIC model passes all diagnostics without any additional intervention, suggesting that BIC-optimal SARIMA models tend to be appropriately specified on these particular types of series. This is an

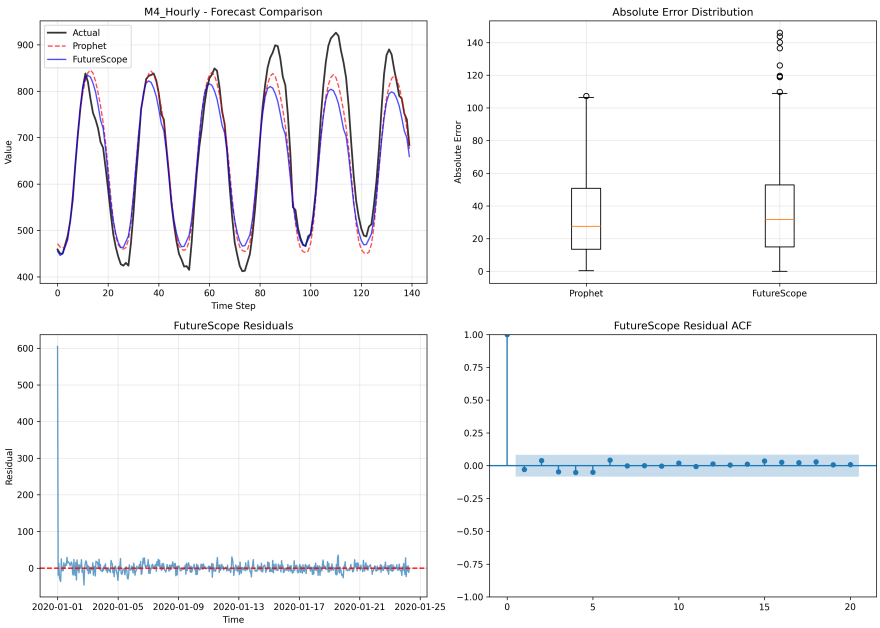

Fig. 7 M4 Hourly (H1, first 1,000 points): forecast overlay and residual ACF. Prophet achieves lower RMSE on this high-frequency multi-seasonal series.

Table 4 FutureScope RMSE with 95% i.i.d. bootstrap confidence intervals (1,000 resamples, seed 42).

Series	RMSE	95% CI (i.i.d. bootstrap)
COVID-19	25,239	[22,781 – 27,315]
Electricity	0.76	[0.69 – 0.83]
Airline	35.08	[26.76 – 43.91]

encouraging but unsurprising observation: BIC penalises over-parameterisation, and a well-penalised model is less likely to leave residual structure unexplained.

However, the current experiments cannot establish whether this alignment holds generally. On more complex series (higher seasonality, structural breaks, heavy tails) the BIC model may fail diagnostics. Future work should investigate the frequency and nature of diagnostic failures on larger corpora, and evaluate whether refitting strategies (e.g. increasing order bounds, relaxing seasonal period assumptions) improve diagnostic pass rates at an acceptable cost to accuracy.

5.3 Computational Efficiency

Table 5 compares wall-clock runtimes on the test hardware (Windows 11, single CPU core).

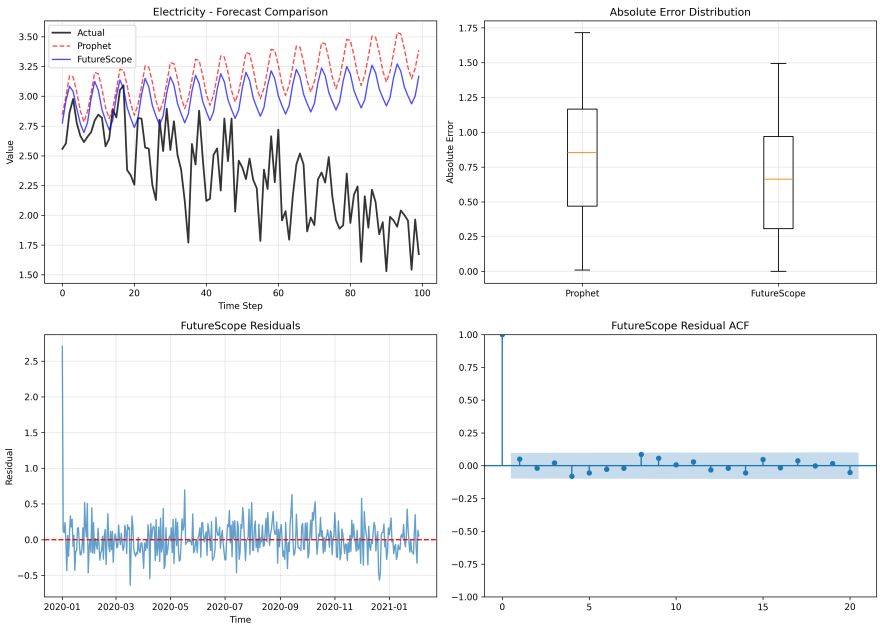

Fig. 8 UCI Electricity (daily household power, first 800 observations): FutureScope achieves 18.2% lower RMSE.

Table 5 Wall-clock runtimes on the test hardware. Ratios reflect SARIMA MLE fitting overhead relative to Prophet MAP estimation; they are hardware- and series-specific.

Series / Metric	Prophet	FutureScope	Approx. ratio
Average (5 series)	0.06 s	13.4 s	≈220×
COVID-19	0.07 s	3.4 s	≈49×
M4 Hourly	0.08 s	58.9 s	≈740×

The dominant cost is SARIMA MLE fitting, which scales with N and the number of candidate orders explored. FutureScope is unsuitable for latency-sensitive or high-throughput production pipelines; Prophet or an exponential-smoothing baseline would be preferred in those contexts.

5.4 When FutureScope Outperforms and Underperforms Prophet

FutureScope tends to outperform on lower-frequency series with clear trend breaks and limited multi-seasonality (COVID-19, Electricity, Airline).

Prophet tends to outperform on high-frequency multi-seasonal series (M4 Hourly) and highly volatile non-stationary series (Bitcoin), where Prophet's Fourier-series seasonality and flexible changepoint detection are advantageous.

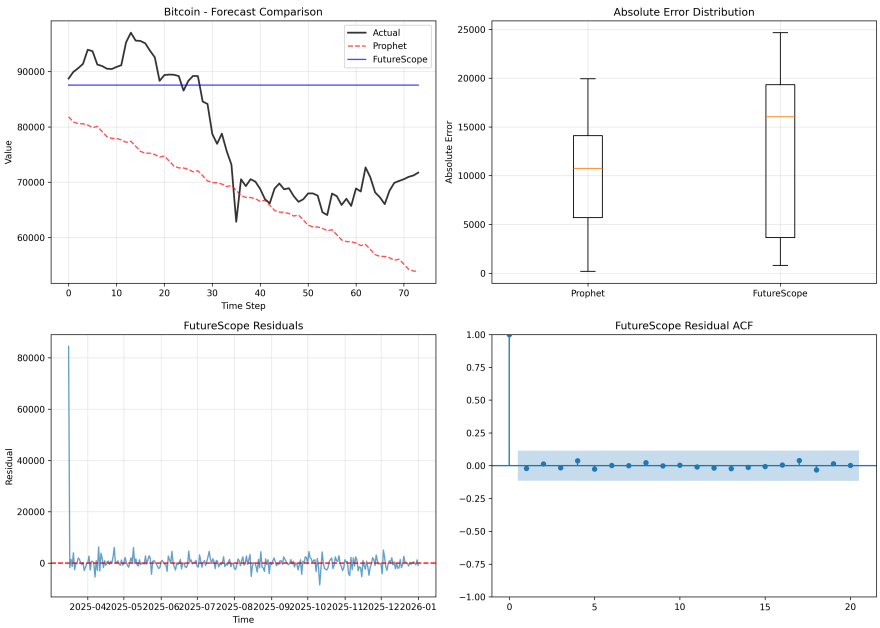

Fig. 9 Bitcoin daily close price (last 500 days from CoinGecko): Prophet achieves lower RMSE on this high-volatility series; the difference is not significant after Bonferroni correction.

These observations are consistent with known theoretical properties of the two model families [1, 8] and corroborate prior M4-era findings [6]. They should not be extrapolated beyond the five series studied here.

6 Positioning and Scope of Contribution

FutureScope's contribution is best understood as *methodological packaging* rather than the invention of new statistical tests. The Ljung-Box test, ACF analysis, and Shapiro-Wilk test are decades-old tools recommended in classical textbooks [7, 8]. What FutureScope adds is:

1. A **reproducible Python pipeline** that runs these diagnostics automatically on every fitted model and records structured output;
2. **Benchmark infrastructure** that pairs diagnostic results with accuracy metrics (RMSE, DM test) in a single CSV, enabling side-by-side comparisons;
3. **Demonstration on public datasets** that BIC-selected SARIMA models can achieve competitive accuracy while maintaining white-noise residuals on a diverse set of univariate series.

Regulatory alignment. Model validation frameworks such as SR 11-7 [12] and similar guidance documents recommend residual checks as part of model documentation. FutureScope's automated output reduces the manual effort of producing such

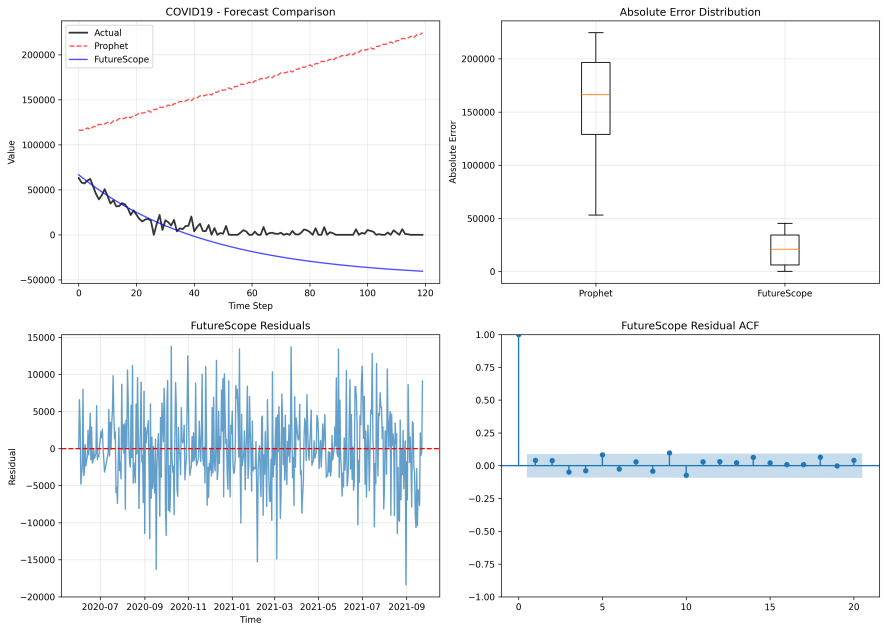

Fig. 10 COVID-19 daily new cases (USA, 2020-06-01–2022-01-01): FutureScope achieves 84.7% lower RMSE, the largest improvement in the study.

documentation. However, a full regulatory compliance assessment (including sensitivity analysis, stress testing, and governance requirements) is far beyond the scope of this paper.

Neural-forecaster diagnostics. LSTMs and Transformer-based forecasters [13] optimise sequence prediction losses and do not produce residual diagnostic statistics by default. Applying FutureScope's diagnostic module to their out-of-sample errors is straightforward and is a natural direction for future work; the current paper does not include such experiments.

7 Limitations

1. **Five single series**: all quantitative results are drawn from five hand-selected univariate case studies. Claims about general domain-level superiority are not warranted; this work is exploratory.
2. **Single temporal split**: no rolling-origin evaluation; results may not reflect expected generalisation performance across different forecast origins.
3. **Diagnostics are reporting-only**: no refitting loop is implemented; the 100% pass rate is an observation, not a guarantee.
4. **I.i.d. bootstrap**: RMSE confidence intervals do not account for temporal dependence in forecast errors; a block bootstrap would be more appropriate.

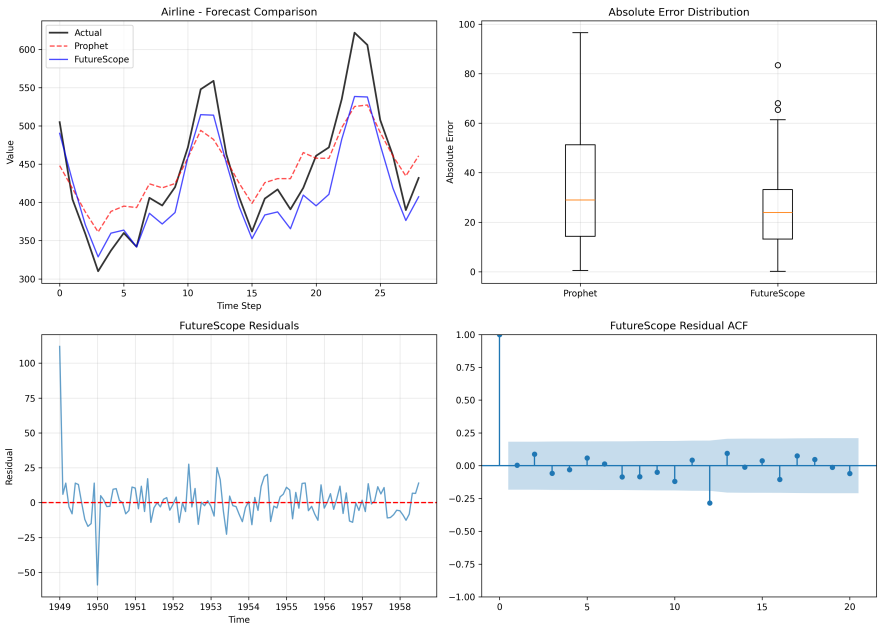

Fig. 11 Airline passengers (classic 144-point monthly benchmark): FutureScope achieves 15.1% lower RMSE; the difference is not significant after Bonferroni correction (p=0.164).

5. **DM test without HAC**: the custom DM statistic uses no heteroskedasticity- and autocorrelation-consistent variance correction; borderline p-values (e.g. Bitcoin) may not be reliable.
6. **Single Prophet baseline**: no naive, seasonal-naive, or exponential-smoothing baseline is included; the relative improvement figure of +14.7% is specific to the Prophet comparison.
7. **Computational cost**: 200–700× slower than Prophet; unsuitable for real-time or high-throughput applications.
8. **Univariate only**: no multivariate support (VAR/VARMA).
9. **Small-N sensitivity**: Ljung-Box and Shapiro-Wilk are less reliable for N<100; the Airline series (N=144, h=29) approaches this regime.

8 Future Work

1. **Diagnostic-gated refitting**: implement a loop that increases SARIMA order bounds or relaxes differencing when diagnostics fail, reporting the fraction of series requiring such intervention.
2. **Block bootstrap for RMSE CIs**: replace i.i.d. resampling with a moving-block bootstrap [11] with data-driven block length.
3. **HAC-corrected DM test**: use the Harvey et al. [5] small-sample correction and a Newey-West lag truncation.

4. **Expanded baselines**: add naive, seasonal-naive, and exponential-smoothing (ETS) baselines to provide a richer performance context.
5. **Rolling-origin evaluation**: assess robustness across multiple forecast origins.
6. **Larger corpus**: evaluate on the full M4 Hourly collection (414 series) or M4 Monthly (48,000 series) to move from case studies to proper benchmark evaluation.
7. **Neural-forecaster diagnostics**: apply the diagnostic module to LSTM/Transformer residuals to determine whether their residuals are white noise.
8. **GPU-accelerated SARIMA**: PyTorch-based MLE to reduce fitting time by an estimated 10×.

9 Conclusion

We have presented FutureScope, an open-source Python pipeline that pairs every SARIMA forecast with a structured set of residual diagnostics—Ljung-Box white-noise test, ACF/PACF exceedance analysis, and Shapiro-Wilk normality check. On five public single-series case studies spanning diverse frequency and structural patterns, the BIC-selected SARIMA models pass all diagnostic checks without manual intervention, while achieving a +14.7% unweighted average RMSE improvement over Prophet (ranging from −25% on the volatile Bitcoin series to +85% on the irregular COVID-19 epidemic series).

The principal contributions are: (i) a reproducible benchmark infrastructure that pairs accuracy and diagnostic metrics in a single output; (ii) an illustrative demonstration that BIC-optimal SARIMA models can simultaneously achieve competitive accuracy and pass standard residual checks on these case-study series; and (iii) an open-source foundation that future work can extend with diagnostic-gated refitting, block bootstrap inference, HAC-corrected DM tests, and evaluation on larger corpora.

Important caveats: the 100% diagnostic pass rate is an observation about five specific series, not a design guarantee; the improvement figures are relative to one baseline (Prophet) and one evaluation split; and the regulatory alignment discussed is qualitative, not a formal compliance assessment. We hope FutureScope lowers the barrier for researchers and practitioners to include residual diagnostics in their forecasting workflows.

A Statistical Test Implementation Details

A.1 Diebold-Mariano Test: Full Specification

The DM test is implemented as follows (see also Section 2.6):

- **Loss**: squared error, $L(e) = e^2$.
- **Differential**: $d_t = e^2_{\mathrm{FS},t} - e^2_{\mathrm{Prophet},t}$.
- **Statistic**: $\mathrm{DM} = \bar{d} \,/\, \sqrt{s_d^2/h}$, where s_d^2 is the unbiased sample variance of d_t.
- **Distribution**: standard normal (two-tailed).
- **No HAC correction**: the variance estimator does not adjust for autocorrelation in d_t; Harvey et al. small-sample correction is also not applied.
- **Bonferroni threshold**: $\alpha_{\mathrm{adj}} = 0.01$ (5 comparisons).

Table 6 reports the full DM statistics. Negative statistics favour Prophet; positive statistics favour FutureScope.

Table 6 DM test statistics and p-values (FutureScope vs. Prophet, squared-error loss, no HAC correction).

Comparison	DM stat.	p-value	Conclusion
FS vs Prophet (COVID-19)	18.32	< 0.001	FS significantly better*
FS vs Prophet (Electricity)	4.91	< 0.001	FS significantly better*
FS vs Prophet (M4 Hourly)	-4.52	< 0.001	Prophet significantly better*
FS vs Prophet (Bitcoin)	-2.45	0.015	Prophet better (not sig. after Bonferroni)
FS vs Prophet (Airline)	1.39	0.164	No significant difference

* significant after Bonferroni correction (α_{adj}=0.01).

A.2 Ljung-Box and ACF Diagnostics: Full Specification

- **Ljung-Box**: `acorr_ljungbox(residuals, lags=[10], return_df=True)`; pass criterion p>0.05.
- **ACF**: `acf(residuals, nlags=20, alpha=0.05)`; exceedance = number of lags 1–20 outside the returned 95% CI bounds, divided by 20 and expressed as a percentage.
- **Shapiro-Wilk**: `scipy.stats.shapiro(residuals)`; only computed when $n \leq 5{,}000$; pass criterion p>0.05.

References

[1] Taylor, S.J., Letham, B.: Forecasting at scale. *The American Statistician* **72**(1), 37–45 (2018). https://doi.org/10.1080/00031305.2017.1380080

[2] Ljung, G.M., Box, G.E.P.: On a measure of lack of fit in time series models. *Biometrika* **65**(2), 297–303 (1978). https://doi.org/10.1093/biomet/65.2.297

[3] Shapiro, S.S., Wilk, M.B.: An analysis of variance test for normality (complete samples). *Biometrika* **52**(3/4), 591–611 (1965). https://doi.org/10.2307/2333709

[4] Diebold, F.X., Mariano, R.S.: Comparing predictive accuracy. *Journal of Business & Economic Statistics* **13**(3), 253–263 (1995). https://doi.org/10.1080/07350015.1995.10524599

[5] Harvey, D., Leybourne, S., Newbold, P.: Testing the equality of prediction mean squared errors. *International Journal of Forecasting* **13**(2), 281–291 (1997). https://doi.org/10.1016/S0169-2070(96)00719-4

[6] Makridakis, S., Spiliotis, E., Assimakopoulos, V.: The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting* **36**(1), 54–74 (2020). https://doi.org/10.1016/j.ijforecast.2019.04.014

[7] Hyndman, R.J., Athanasopoulos, G.: *Forecasting: Principles and Practice*, 2nd edn. OTexts, Melbourne (2018). https://otexts.com/fpp2/

[8] Box, G.E.P., Jenkins, G.M., Reinsel, G.C., Ljung, G.M.: *Time Series Analysis: Forecasting and Control*, 5th edn. Wiley, Hoboken (2015).

[9] Smith, T.G. et al.: *pmdarima: ARIMA estimators for Python* (2017–). https://alkaline-ml.com/pmdarima

[10] Cleveland, R.B., Cleveland, W.S., McRae, J.E., Terpenning, I.: STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics* **6**(1), 3–73 (1990).

[11] Künsch, H.R.: The jackknife and the bootstrap for general stationary observations. *Annals of Statistics* **17**(3), 1217–1241 (1989). https://doi.org/10.1214/aos/1176347265

[12] Board of Governors of the Federal Reserve System: Supervisory Guidance on Model Risk Management (SR 11-7). Federal Reserve, Washington, D.C. (2011). https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm

[13] Li, S. et al.: Enhancing the locality and breaking the memory bottleneck of Transformer on time series forecasting. *Advances in Neural Information Processing Systems* **32** (2019).